Continuação da Aula 02 – Introdução à POO

Estes exercícios são na verdade um "LAB", onde você executará diversos exemplos que demonstram os termos e ideias fundamentais da POO apresentados na Aula 02. Todos os exemplos foram retirados ou adaptados do seguinte tutorial: https://www.python-course.eu/python3_object_oriented_programming.php.

EXEMPLO 1 – sintaxe básica

```
class Robot:
    pass

x = Robot()
y = Robot()
y2 = y
print(y is y2)
print(y is x)

>>> %Run robot.py
True
False
```

Explicação: Nesse exemplo, definimos a classe Robot em apenas duas linhas de código, sem nenhum atributo ou método. Depois instanciamos dois diferentes robôs x e y. Além disso, criamos uma referência (shallow copy) y2 para y, ou seja, os nomes y2 e y apontam para o mesmo objeto.

EXEMPLO 2 – criação dinâmica de atributos (*não é feita na prática!*)

```
class Robot:
    pass

x = Robot()
x.nome = "Zed"
x.ano_fabricacao = "2020"

y = Robot()
y.nome = "Gort"
y.ano_fabricacao = "2008"


print(x.__dict__)
print(y.__dict__)

>>> %Run robot.py
{'nome': 'Zed', 'ano_fabricacao': '2020'}
{'nome': 'Gort', 'ano_fabricacao': '2008'}
```

Explicação: como Python é uma linguagem dinâmica, podemos modificar um objeto em tempo de execução. Neste exemplo, fizemos isso juntando um nome arbitrário ao nome da instância, separado por um ponto "." e, em seguida, fazendo uma atribuição. Assim, os robôs passaram a ter nome e ano de fabricação.

O problema desse código é que, como os atributos foram criados dinamicamente, não é possível garantir que todo robô tenha um nome e um ano de fabricação. É preciso definir uma forma padronizada que permita com que possamos instanciar objetos da classe Robot com determinadas configurações iniciais (sempre com nome e ano de fabricação, por exemplo). Isso pode ser feito com o uso do método construtor, conforme mostra o exemplo a seguir.

EXEMPLO 3 – construtor

```
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if ano_fabr==None:
            self.ano_fabricacao = 2020
        else:
            self.ano_fabricacao = ano_fabr

x = Robot("Zed")
y = Robot("Gort", 2008)

print(x.__dict__)
print(y.__dict__)

>>> %Run robot.py
{'nome': 'Zed', 'ano_fabricacao': '2020'}
{'nome': 'Gort', 'ano_fabricacao': '2008'}
```

Explicação: agora é o __init__ (método construtor) que vai inicializar o objeto. Todo robô precisa ter pelo menos um nome para ser criado. Já o ano de fabricação é opcional: caso não seja passado, o construtor atribuirá o valor 2020. Se você tentar criar um robô sem nome, ocorrerá o seguinte erro:

```
>>>  z = Robot()
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
TypeError: __init__() missing 1 required positional argument: 'nome'
```

EXEMPLO 4 – atributos de classe

```
class Robot:

    tipo = "lata velha"

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if ano_fabr==None:
            self.ano_fabricacao = 2020
        else:
            self.ano_fabricacao = ano_fabr

x = Robot("Zed")
y = Robot("Gort", 2008)

print(x.__dict__)
print(y.__dict__)

print("\n")
print(x.tipo, y.tipo, Robot.tipo)

print("\n")
print(Robot.__dict__)
```

```
>>> %Run robot.py
{'nome': 'Zed', 'ano_fabricacao': 2020}
{'nome': 'Gort', 'ano_fabricacao': 2008}

lata velha lata velha lata velha

{'__module__': '__main__', 'tipo': 'lata velha', '__init__': <function Robot.__init__ at 0x03D07978>, '__dict__': <attribute '__dict__' of 'Robot' objects>, '__weakref__': <attribute '__weakref__' of 'Robot' objects>, '__doc__': None}
```

Explicação: No anterior (Exemplo 3), havíamos mostrado como trabalhar com os atributos de instância, que são manipulados dentro da classe com o uso de self e são específicos de cada objeto. No entanto, também podemos definir atributos de classe, que sempre terão o mesmo valor para qualquer objeto instanciado da classe.

Neste exemplo, definimos o atributo "tipo" como um atributo de classe. Isso significa que independentemente da instância, todo robô será 'lata velha'. Veja na linha de código `print(x.tipo, y.tipo, Robot.tipo)` que podemos imprimir o valor do "tipo" de um robô usando Robot.tipo, além de x.tipo e y.tipo.

Note ainda, na última linha de código do programa, que a própria classe Robot também possui um atributo mágico __dict__ e que "tipo" aparece nele. Se, por exemplo, tentamos acessar x.tipo, o Python checa primeiro se "tipo" é uma chave do dicionário x.__dict__.

Caso não seja, ele checa se é uma chave de Robot.__dict__ (se não for de nenhuma das duas, ocorre um erro como o mostrado no Exemplo 3).

EXEMPLO 5 – criando um método fora da classe (*não é feita na prática!*)

```
def falar_oi(obj):
    print("oi, me chamo " + obj.nome + "!")

class Robot:

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if ano_fabr==None:
            self.ano_fabricacao = 2020
        else:
            self.ano_fabricacao = ano_fabr


x = Robot("Zed")
falar_oi(x)
```

```
>>> %Run robot.py
oi, me chamo Zed!
```

Explicação: criamos uma função "falar_oi" que recebe um objeto como parâmetro e imprime uma mensagem (assumindo que o objeto passado como parâmetro possui um atributo chamado "nome"). Veja que o código funcionou, mas esta não é a forma mais apropriada de implementar uma ação para uma classe. A função "falar_oi" é específica para objetos do tipo Robô... se passarmos para ela um outro tipo de objeto (uma lista, por exemplo), que não possui o atributo "nome", ocorrerá um erro na execução do programa.

```
>>> falar_oi([1,2,3])
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
  File "D:\ENCE\ProgAvancada\aulas\progs\oop\robot.py", line 2, in
falar_oi
    print("oi, me chamo " + obj.nome + "!")
AttributeError: 'list' object has no attribute 'nome'
```

Desta forma, a maneira adequada de implementar as ações de um objeto é a seguinte: em vez de definir uma função fora da classe, devemos definir o método diretamente dentro da definição da classe. É o que mostra o próximo exemplo.

EXEMPLO 6 – criando um método

```
class Robot:

    def __init__(self, nome, ano_fabr=None):
```

```
        self.nome = nome
        if ano_fabr==None:
            self.ano_fabricacao = 2020
        else:
            self.ano_fabricacao = ano_fabr

    def falar_oi(self):
        print("oi, me chamo " + self.nome + "!")


x = Robot("Zed")
x.falar_oi()

y = Robot("Gort", 2008)
y.falar_oi()

>>> %Run robot.py
oi, me chamo Zed!
oi, me chamo Gort!
```

Explicação: um método é apenas uma função definida dentro da classe. Na definição do método, o primeiro parâmetro deve ser sempre self, que significa uma referência para a instância que está chamado o método.

Dessa forma, na chamada `x.falar_oi()` self corresponde ao objeto x e na chamada `y.falar_oi()` self corresponde ao objeto y. Ao chamar o método, não precisamos passar self, pois o Python faz isso automaticamente para a gente, de forma escondida.

Também é possível chamar um método da forma a seguir (que é pouco utilizada na prática):

```
>>> Robot.falar_oi(x)
oi, me chamo Zed!
```

Em resumo, um método difere de uma função em apenas dois aspectos:

- Ele pertence a uma classe e é definido dentro da mesma.
- O primeiro parâmetro da definição de um método precisa ser uma referência para a instância que chamou o método. Esse parâmetro é normalmente chamado de "self" (curiosamente, "self" não é uma palavra reservada do Python, é apenas uma convenção).

EXEMPLO 7 – criando um método com parâmetro (além do self...)

```
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if ano_fabr==None:
            self.ano_fabricacao = 2020
        else:
```

```
            self.ano_fabricacao = ano_fabr

    def falar_oi(self, personalidade):
        if personalidade.upper() == "BOM":
            print("Oi, sou " + self.nome + " e vim em missão de paz!")
        elif personalidade.upper() == "MAU":
            print("Oi, sou " + self.nome + " e vim te escravizar!")
        else:
            print("Oi, sou " + self.nome + " e esqueci o que vim fazer!")


x = Robot("Zed")
x.falar_oi("BOM")

y = Robot("Gort", 2008)
y.falar_oi("MAU")

z = Robot("Torg", 2008)
z.falar_oi("SEI LÁ")

>>> %Run robot.py
Oi, sou Zed e vim em missão de paz!
Oi, sou Gort e vim te escravizar!
Oi, sou Torg e esqueci o que vim fazer!
```

Explicação: Neste exemplo, alteramos o método "falar_oi", adicionando um outro parâmetro além do self, denominado "personalidade". Dependendo do valor passado para a personalidade, nosso robô poderá falar uma frase amigável ou não...

EXEMPLO 8 – troca de mensagens

```
class Robot:

    def __init__(self, nome, ano_fabr=None):
        self.nome = nome
        if ano_fabr==None:
            self.ano_fabricacao = 2020
        else:
            self.ano_fabricacao = ano_fabr

    def convoca(self, outro_robo):
        print(self.nome + " convocando " + outro_robo.nome + ":")
        print("É hora de destruir os humanos!!! ")


x = Robot("Zed")
y = Robot("Gort", 2008)

x.convoca(y)

>>> %Run robot.py
Zed convocando Gort:
É hora de destruir os humanos!!!
```

Explicação: um sistema orientado a objetos é formado por diferentes objetos, que podem ser de diferentes classes, e que se comunicam trocando informações através de seus métodos (processo chamado de "troca de mensagens" entre objetos).

Neste exemplo, o método "convoca" exemplifica um caso de troca de mensagens entre objetos. Esse método é definido com dois parâmetros. O primeiro não é nenhuma novidade, é o self de sempre. Já o segundo parâmetro, chamado "outro_robo" deve receber um outro objeto do tipo Robot, com o qual "self" irá se comunicar. (nesse caso, literalmente, *self convoca outro robô para destruir os humanos*).

Ao longo das próximas aulas mostraremos mais exemplos de utilização de métodos e troca de mensagens (inclusive entre objetos de diferentes classes). Mas por hoje é só!